

Logistic Regression on MNIST with Quantization

Part 1: Set Up and Data Preparation

We utilized the MNIST dataset, a widely used dataset for handwritten digit recognition, to implement and evaluate a logistic regression model. The dataset consists of 60,000 images, each of size 28x28, representing digits from 0 to 9. To prepare the data, we applied the following steps:

- Loaded the MNIST dataset using `torchvision.datasets`.
- Normalized the images using the mean and standard deviation for the MNIST dataset.
- Flattened each image to a 1D tensor of size 784 (28x28).
- Split the data into training (80%) and test (20%) sets using `train_test_split`.

Part 2: Model Building

The logistic regression model was built using PyTorch. The model comprises:

- An input layer of 784 nodes (corresponding to the flattened image size).
- An output layer of 10 nodes (one for each digit class). We defined the loss function as `CrossEntropyLoss` and the optimizer as `Stochastic Gradient Descent (SGD)` with a learning rate of 0.01.

Part 3: Training the Model

The model was trained over 10 epochs, and the loss was printed at each epoch to monitor the model's progress. The forward pass involved predicting the digit for each input image, calculating the loss, and backpropagating the error to update the model's parameters using the optimizer.

After training, the model was evaluated on the test set to calculate the test accuracy. Additionally, the original model size was saved for later comparison with the quantized version.

Results after Training

- **Test Accuracy:** 68.49%
- **Original Model Size:** 32.19 KB

Part 4: Quantization

To optimize the model for efficiency, we applied a custom quantization function that scaled the model's weights to lower precision (8-bit) without significantly impacting performance. The quantization was performed by rounding the weights after multiplying them by a scale factor. This resulted in a smaller model size, reducing the storage and computational requirements.

Part 5: Inference and Evaluation of the Quantized Model

We implemented an inference function for both the original and quantized models. The inference time was measured for both models on the test data, and the accuracy of the quantized model was computed to compare with the original.

Inference Time

- **Original Model Inference Time:** 34.60 ms
- **Quantized Model Inference Time:** 10.93 ms

Quantized Model Evaluation

- **Quantized Model Accuracy:** 68.49%
- **Quantized Model Size:** 32.20 KB

Part 6: Comparison Results

The results of the original and quantized models were compared in terms of accuracy, inference time, and model size.

Metric	Original Model	Quantized Model
Test Accuracy	68.51%	68.49%
Inference Time (ms)	32.60 ms	10.93 ms
Model Size (KB)	32.19 KB	32.20 KB

Conclusion

The quantized model achieved nearly the same accuracy as the original model but with significantly reduced model size and inference time. This demonstrates the effectiveness of quantization for making models more efficient, especially for deployment on resource-constrained devices like mobile phones and embedded systems.